

Another program on Variable Shadow :

```
class Test
{
    void main()
    {
        Student s = new Student();
        s.setMarks(90);
    }
}
```

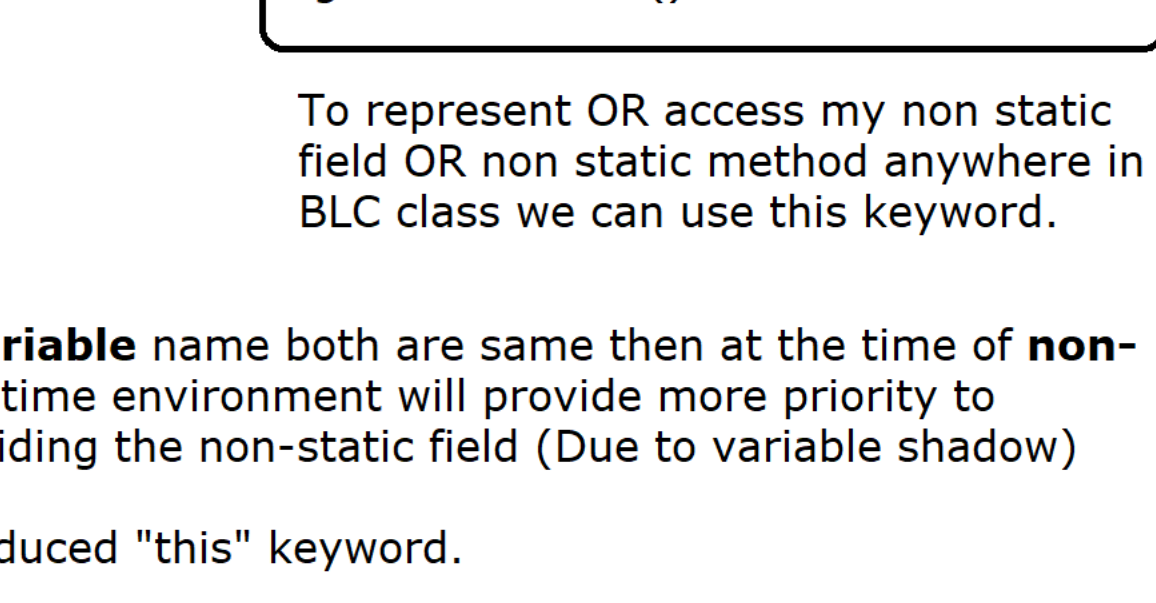
```
class Student
{
    int marks = 80;

    public void setMarks(int marks)
    {
        IO.println(marks); //90
        IO.println(this.marks); //80
    }
}
```

this keyword in Java :

Diagram for ELC class

```
public class ThisKeywordDemo
{
    public static void main(String[] args)
    {
        Product p1 = new Product();
        p1.setProductData(101, "Laptop", 95000);
        p1.getProductData();
    }
}
```



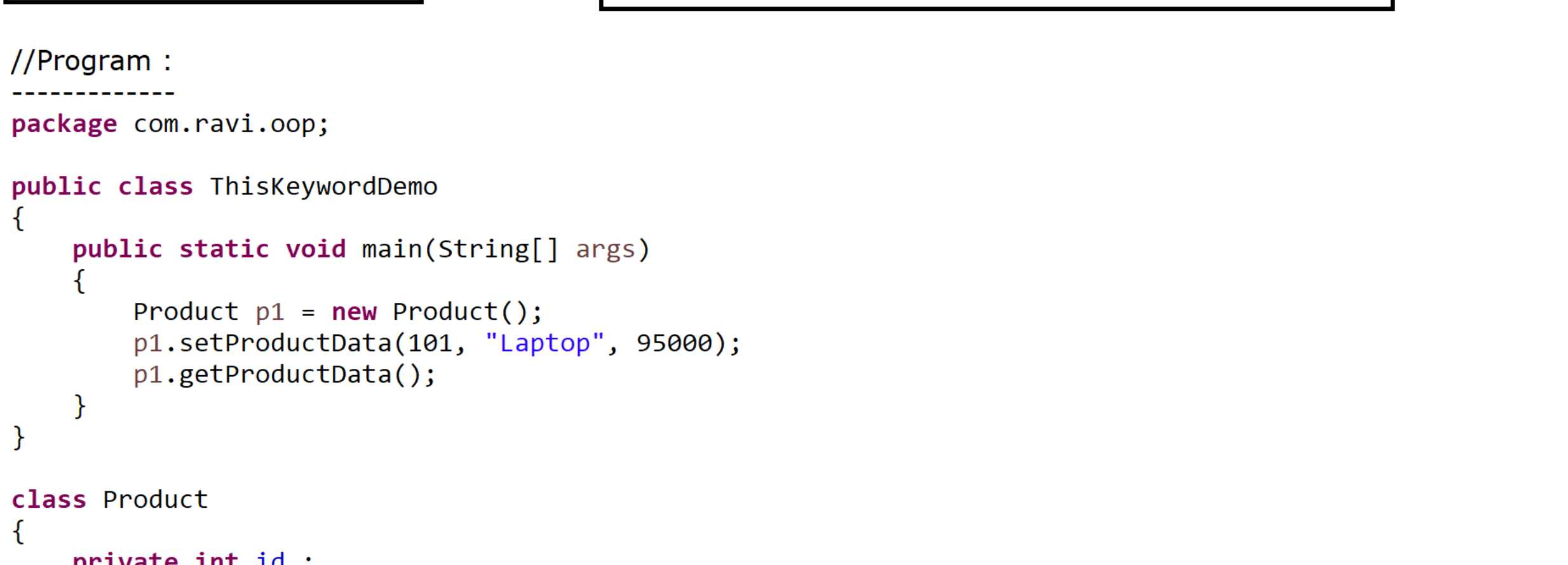
Whenever **non static field name** and **parameter/local variable** name both are same then at the time of **non-static field** initialization through parameter variable our runtime environment will provide more priority to parameter variable/local variable, parameter variables are hiding the non-static field (Due to variable shadow)

To avoid the above said problem, Java software people introduced "this" keyword.

this keyword **refers to the current object** (as shown in the diagram) and we know that non static fields are the part of the object so to represent non static field & non static method anywhere in the BLC class we can use this keyword.

this keyword represents non static members so, we cannot use this keyword from static context OR static area like static method, static block OR static nested class.

Java compiler will automatically add this keyword to each and every non static method as well as constructor as a first parameter and it is final by default.



```
//Program :
package com.ravi.oop;

public class ThisKeywordDemo
{
    public static void main(String[] args)
    {
        Product p1 = new Product();
        p1.setProductData(101, "Laptop", 95000);
        p1.getProductData();
    }
}

class Product
{
    private int id ;
    private String name;
    private double price;

    public void setProductData(int id, String name, double price)
    {
        this.id = id;
        this.name = name;
        this.price = price;
    }

    public void getProductData()
    {
        IO.println("Product Id is :"+id);
        IO.println("Product Name is :"+name);
        IO.println("Product Price is :"+price);
    }
}
```

Constructor

* Constructor plays a **major role in object creation** that is the reason it is known as Constructor.

Q) What is the advantage of writing constructor in our class :

Diagram :

```
Employee.java
public class Employee
{
    private int id;

    public Employee() //default no argument constructor added by java compiler
    {
    }

    public void setId(int id)
    {
        this.id = id;
    }
}

Employee scott = new Employee();
scott.setId(101);
```

this.id = 0
this.id = 101

* In the above diagram non static field initialization (this.id = 0) is done at the time of creating the object but non static field re-initialization (this.id = 101) is done at the time of calling the setId() method so, the conclusion is, non static field initialization (this.id = 0) and non static field re-initialization (this.id = 101) both are done in two different lines

* If a developer will write own user-defined constructor then non static field initialization (this.id = 0) and non static field re-initialization (this.id = 101) both **will be done in the same line during the creation of the object as shown in the diagram**

Example with user-defined constructor :

```
public class Employee
{
    private int id;

    public Employee(int id) //User defined constructor
    {
        this.id = id;
    }
}

Employee scott = new Employee(101);
```

id = 101

Example 1:

Initializing the non static fields using non static method [old approach]

```
class Test
{
    void main()
    {
        Employee e1 = new Employee();
        e1.setId(101);
        e1.show();
    }
}

class Employee
{
    private int id;

    public void setId(int id)
    {
        this.id = id;
    }

    public void show()
    {
        IO.println("Employee id is :"+this.id);
    }
}
```

Example 2 :

Initializing the non static field using user-defined constructor [new Approach]

```
class Test
{
    void main()
    {
        Employee e1 = new Employee(101);
        e1.show();
    }
}

class Employee
{
    private int id;

    public Employee(int id)
    {
        this.id = id; //Modifying the default value with user value
    }

    public void show()
    {
        IO.println("Employee id is :"+this.id);
    }
}
```

Some important points of constructor :

It is used to construct the object so it is called Constructor.

If the name of the class and name of the method both are exactly same and It should not contain any return type then it is called constructor.

**The main purpose of constructor to initialize the object properties (non-static fields) with user-defined value.

Every class must contain at-least one constructor either implicitly added by compiler or explicitly written by user.

Every time we create an object in java by using new keyword, at-least one constructor must be invoked.

A constructor never contains any return type including void also.

```
class Test
{
    void main()
    {
        Employee e1 = new Employee();
        IO.println("Program completed");
    }
}

class Employee
{
    public Employee()
    {
        IO.println("Parameter less constructor");
        return; //Valid [Will return control only]
    }
}
```

A constructor is automatically called and executed at the time of creating the object.